Teoria Współbieżności

Kajetan Frątczak 21.11.2025 r.

Laboratorium 06

Badanie efektywności

1. **Treść zadań**

Zadanie 1

Problem czytelników i pisarzy

Problem czytelników i pisarzy proszę rozwiązać przy pomocy: semaforów i zmiennych warunkowych. Proszę wykonać pomiary dla różnej ilości czytelników (10-100) i pisarzy (od 1 do 10). W sprawozdaniu proszę narysować 3D wykres czasu w zależności od liczby wątków i go zinterpretować.

Zadanie 2

Blokowanie drobnoziarniste

1. Zamek (lock) jest przydatny wtedy, gdy operacje zamykania/otwierania nie mogą być umieszczone w jednej metodzie lub bloku synchronized. Przykładem jest zakładanie blokady (lock) na elementy struktury danych, np. listy. Podczas przeglądania listy stosujemy następujący algorytm:
1. zamknij zamek na pierwszym elemencie listy
2. zamknij zamek na drugim elemencie
3. otwórz zamek na pierwszym elemencie
4. zamknij zamek na trzecim elemencie
5. otwórz zamek na drugim elemencie
6. powtarzaj dla kolejnych elementów

Dzięki temu unikamy konieczności blokowania całej listy i wiele wątków może równocześnie przeglądać i modyfikować różne jej fragmenty.

Ćwiczenie - blokowanie drobnoziarniste

1. Proszę zaimplementować listę, w której każdy węzeł składa się z wartości typu Object, referencji do następnego węzła oraz zamka (lock).
2. Proszę zastosować metodę drobnoziarnistego blokowania do następujących metod listy:

boolean contains(Object o); //czy lista zawiera element o
boolean remove(Object o); //usuwa pierwsze wystąpienie elementu o
boolean add(Object o); //dodaje element o na końcu listy

3. Proszę porównać wydajność tego rozwiązania w stosunku do listy z jednym zamkiem blokującym dostęp do całości. Należy założyć, że koszt czasowy operacji na elemencie listy (porównanie, wstawianie obiektu) może być duży - proszę wykonać pomiary dla różnych wartości tego kosztu.

2. Argumentacja rozwiązania

Zadanie 1

Implementacja z semaforami: Wykorzystałem klasyczne rozwiązanie problemu czytelników i pisarzy z preferencją dla czytelników. Semafory writeLock i mutex kontrolują dostęp do zasobu wspólnego. Pierwszy czytelnik blokuje pisarzy (writeLock.acquire()), a ostatni zwalnia pisarzy (writeLock.release()). Semafor mutex chroni licznik readCount przed jednoczesnym dostępem. Taka implementacja pozwala wielu czytelnikom na równoczesny dostęp, ale może prowadzić do zagłodzenia pisarzy przy dużej liczbie czytelników.
Implementacja ze zmiennymi warunkowymi: W tym rozwiązaniu zastosowałem preferencję dla pisarzy, aby zminimalizować ich czas oczekiwania. Wykorzystałem ReentrantLock oraz dwa obiekty Condition: canRead i canWrite. Czytelnicy czekają, jeśli są aktywni pisarze lub oczekują pisarze (waitingWriters > 0). Pisarze są budzeni w pierwszej kolejności (canWrite.signal()), dopiero przy ich braku budzeni są wszyscy czytelnicy (canRead.signalAll()). To rozwiązanie jest bardziej sprawiedliwe dla pisarzy, ale może prowadzić do zagłodzenia czytelników przy intensywnym zapisie.
Pomiary: Przeprowadziłem testy dla liczby czytelników od 10 do 100 i pisarzy od 1 do 10. Symulacja czytania trwa 50ms, a pisania 100ms. Mierzony był całkowity czas wykonania oraz średni czas oczekiwania wątków.

Zadanie 2

Implementacja Fine-Grained: Zastosowałem technikę "hand-over-hand locking" (blokowanie ręka-w-rękę), gdzie każdy węzeł listy posiada własny zamek. Podczas przechodzenia przez listę trzymam jednocześnie dwa zamki: na węźle poprzednim (pred) i bieżącym (curr). Dopiero po zablokowaniu kolejnego węzła zwalniam zamek na poprzednim. Ta technika pozwala wielu wątkom równocześnie operować na różnych fragmentach listy, co zwiększa współbieżność.
Implementacja Coarse-Grained: Lista posiada jeden globalny zamek, który blokuje dostęp do całej struktury. To rozwiązanie jest prostsze w implementacji i ma mniejszy overhead, ale każda operacja blokuje całą listę, co drastycznie ogranicza współbieżność przy wielu wątkach.
Symulacja kosztownych operacji: Stworzyłem klasę ExpensiveObject, której metoda equals() symuluje kosztowne porównanie przez opóźnienie o zadaną liczbę milisekund. Pozwoliło to zbadać zachowanie obu implementacji przy różnym koszcie operacji na elementach (0, 1, 5, 10, 20 ms).
Pomiary: Testy przeprowadzałem dla 1, 2, 4, 8 i 16 wątków, każdy wykonujący 50 losowych operacji (mix contains, add, remove) na liście o początkowym rozmiarze 20 elementów.

3. Rozwiązanie

Zadanie 1

```java
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Semaphore;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class ReadersWriters {
    // IMPLEMENTACJA Z SEMAFORAMI
    static class SemaphoreReadersWriters {
        private final Semaphore writeLock = new Semaphore(1);
        private final Semaphore mutex = new Semaphore(1);
        private int readCount = 0;

        class Reader extends Thread {
            private long waitingTime;

            @Override
```

```java
        public void run() {
            long startWait = System.currentTimeMillis();
            try {
                mutex.acquire();
                readCount++;
                if (readCount == 1) {
                    writeLock.acquire();
                }
                mutex.release();

                waitingTime = System.currentTimeMillis() - startWait;

                // Symulacja czytania
                Thread.sleep(50);

                mutex.acquire();
                readCount--;
                if (readCount == 0) {
                    writeLock.release();
                }
                mutex.release();

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        public long getWaitingTime() {
            return waitingTime;
        }
    }

    class Writer extends Thread {
        private long waitingTime;

        @Override
        public void run() {
            long startWait = System.currentTimeMillis();
            try {
                writeLock.acquire();

                waitingTime = System.currentTimeMillis() - startWait;

                // Symulacja pisania
                Thread.sleep(100);

                writeLock.release();

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        public long getWaitingTime() {
            return waitingTime;
        }
    }

    public void reset() {
        readCount = 0;
    }
}

// IMPLEMENTACJA ZE ZMIENNYMI WARUNKOWYMI
static class CondVarReadersWriters {
    private final Lock lock = new ReentrantLock();
    private final Condition canRead = lock.newCondition();
```

```java
    private final Condition canWrite = lock.newCondition();
    private int readers = 0;
    private int writers = 0;
    private int waitingWriters = 0;

    class Reader extends Thread {
        private long waitingTime;

        @Override
        public void run() {
            long startWait = System.currentTimeMillis();
            try {
                lock.lock();
                try {
                    // Czekaj jeśli są pisarze lub czekają pisarze (priorytet dla pisarzy)
                    while (writers > 0 || waitingWriters > 0) {
                        canRead.await();
                    }
                    readers++;
                    waitingTime = System.currentTimeMillis() - startWait;
                } finally {
                    lock.unlock();
                }

                // Symulacja czytania
                Thread.sleep(50);

                lock.lock();
                try {
                    readers--;
                    if (readers == 0) {
                        canWrite.signal(); // Obudź pisarza
                    }
                } finally {
                    lock.unlock();
                }

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        public long getWaitingTime() {
            return waitingTime;
        }
    }

    class Writer extends Thread {
        private long waitingTime;

        @Override
        public void run() {
            long startWait = System.currentTimeMillis();
            try {
                lock.lock();
                try {
                    waitingWriters++;
                    // Czekaj jeśli są czytelnicy lub inny pisarz
                    while (readers > 0 || writers > 0) {
                        canWrite.await();
                    }
                    waitingWriters--;
                    writers++;
                    waitingTime = System.currentTimeMillis() - startWait;
                } finally {
                    lock.unlock();
                }
```

```
                // Symulacja pisania
                Thread.sleep(100);

                lock.lock();
                try {
                    writers--;
                    // Priorytet dla pisarzy
                    if (waitingWriters > 0) {
                        canWrite.signal();
                    } else {
                        canRead.signalAll(); // Obudź wszystkich czytelników
                    }
                } finally {
                    lock.unlock();
                }

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        public long getWaitingTime() {
            return waitingTime;
        }
    }

    public void reset() {
        readers = 0;
        writers = 0;
        waitingWriters = 0;
    }
}
```

+ testy, ale wyszłoby za dużo kodu

Zadanie 2

```
// ==================== FINE-GRAINED LIST ====================
class FineGrainedList {
    private class Node {
        Object value;
        Node next;
        Lock lock = new ReentrantLock();

        Node(Object value) {
            this.value = value;
        }
    }

    private final Node head;

    public FineGrainedList() {
        head = new Node(null);
    }

    public boolean contains(Object o) {
        if (o == null) return false;
        Node pred = head;
        pred.lock.lock();
        try {
            Node curr = pred.next;
            if (curr == null) return false;

            curr.lock.lock();
            try {
                while (curr != null) {
```

```
                if (o.equals(curr.value)) {
                    return true;
                }
                Node next = curr.next;
                if (next == null) {
                    return false;
                }
                next.lock.lock();
                pred.lock.unlock();
                pred = curr;
                curr = next;
            }
            return false;
        } finally {
            curr.lock.unlock();
            pred.lock.unlock();
        }
    } catch (Exception e) {
        pred.lock.unlock();
        throw e;
    }
}

public boolean add(Object o) {
    Node pred = head;
    pred.lock.lock();
    try {
        Node curr = pred.next;
        while (curr != null) {
            curr.lock.lock();
            pred.lock.unlock();
            pred = curr;
            curr = curr.next;
        }
        pred.next = new Node(o);
        return true;
    } finally {
        pred.lock.unlock();
    }
}

public boolean remove(Object o) {
    if (o == null) return false;
    Node pred = head;
    pred.lock.lock();
    try {
        Node curr = pred.next;
        if (curr == null) return false;

        curr.lock.lock();
        try {
            while (curr != null) {
                if (o.equals(curr.value)) {
                    pred.next = curr.next;
                    return true;
                }
                Node next = curr.next;
                if (next == null) {
                    return false;
                }
                next.lock.lock();
                pred.lock.unlock();
                pred = curr;
                curr = next;
            }
            return false;
        } finally {
```

```java
                curr.lock.unlock();
                pred.lock.unlock();
            }
        } catch (Exception e) {
            pred.lock.unlock();
            throw e;
        }
    }

    public int size() {
        int count = 0;
        Node pred = head;
        pred.lock.lock();
        try {
            Node curr = pred.next;
            while (curr != null) {
                curr.lock.lock();
                pred.lock.unlock();
                pred = curr;
                count++;
                curr = curr.next;
            }
        } finally {
            pred.lock.unlock();
        }
        return count;
    }
}

// ==================== COARSE-GRAINED LIST ====================
class CoarseGrainedList {
    private class Node {
        Object value;
        Node next;

        Node(Object value) {
            this.value = value;
        }
    }

    private final Node head;
    private final Lock lock = new ReentrantLock();
    public CoarseGrainedList() {
        head = new Node(null);
    }
    public boolean contains(Object o) {
        lock.lock();
        try {
            Node current = head.next;
            while (current != null) {
                if (o.equals(current.value)) return true;
                current = current.next;
            }
            return false;
        } finally {
            lock.unlock();
        }
    }
    public boolean remove(Object o) {
        lock.lock();
        try {
            Node current = head;
            while (current.next != null) {
                if (o.equals(current.next.value)) {
                    current.next = current.next.next;
                    return true;
                }
```

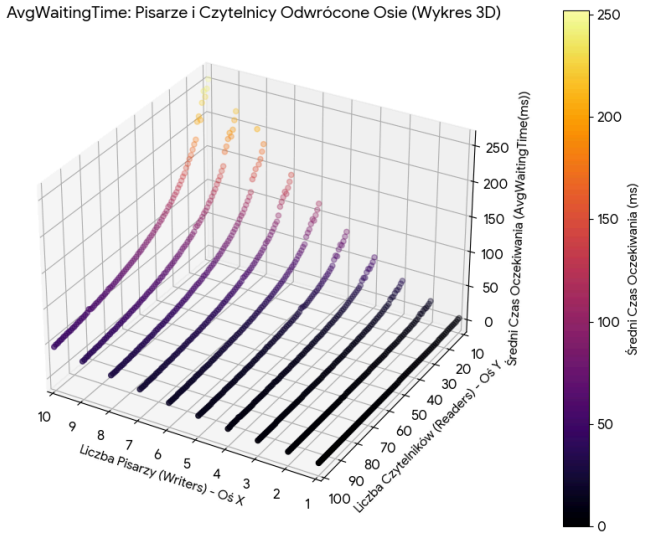
```java
                current = current.next;
            }
            return false;
        } finally {
            lock.unlock();
        }
    }
    public boolean add(Object o) {
        lock.lock();
        try {
            Node current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = new Node(o);
            return true;
        } finally {
            lock.unlock();
        }
    }

    public int size() {
        lock.lock();
        try {
            int count = 0;
            Node current = head.next;
            while (current != null) {
                count++;
                current = current.next;
            }
            return count;
        } finally {
            lock.unlock();
        }
    }
}
```

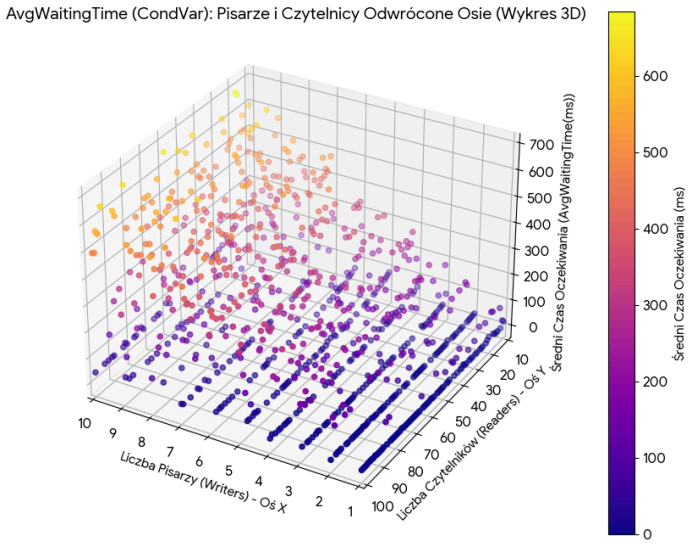
+ **testy, ale wyszłoby za dużo kodu**

4. Wykresy, tabele, wyniki

Zadanie 1



Rysunek 1: Wykres 3D dla semaforów



Rysunek 2: Wykres 3D dla zmiennych warunkowych

Zadanie 2

```
======================================================================
PORÓWNANIE WYDAJNOŚCI LIST
Fine-Grained vs Coarse-Grained Locking
======================================================================

----------------------------------------------------------------------
Koszt operacji: 0 ms
----------------------------------------------------------------------
Testowanie z 1 wątkami... Fine: 6 ms, Coarse: 2 ms, Przyspieszenie: 0,33x
Testowanie z 2 wątkami... Fine: 2 ms, Coarse: 0 ms, Przyspieszenie: 0,00x
Testowanie z 4 wątkami... Fine: 5 ms, Coarse: 2 ms, Przyspieszenie: 0,40x
Testowanie z 8 wątkami... Fine: 7 ms, Coarse: 2 ms, Przyspieszenie: 0,29x
Testowanie z 16 wątkami... Fine: 19 ms, Coarse: 5 ms, Przyspieszenie: 0,26x

----------------------------------------------------------------------
Koszt operacji: 1 ms
----------------------------------------------------------------------
Testowanie z 1 wątkami... Fine: 994 ms, Coarse: 960 ms, Przyspieszenie: 0,97x
Testowanie z 2 wątkami... Fine: 1712 ms, Coarse: 2453 ms, Przyspieszenie: 1,43x
Testowanie z 4 wątkami... Fine: 2355 ms, Coarse: 6426 ms, Przyspieszenie: 2,73x
Testowanie z 8 wątkami... Fine: 4180 ms, Coarse: 18005 ms, Przyspieszenie: 4,31x
Testowanie z 16 wątkami... Fine: 5660 ms, Coarse: 55711 ms, Przyspieszenie: 9,84x

----------------------------------------------------------------------
Koszt operacji: 5 ms
----------------------------------------------------------------------
Testowanie z 1 wątkami... Fine: 3079 ms, Coarse: 3087 ms, Przyspieszenie: 1,00x
Testowanie z 2 wątkami... Fine: 5632 ms, Coarse: 7667 ms, Przyspieszenie: 1,36x
Testowanie z 4 wątkami... Fine: 7731 ms, Coarse: 18968 ms, Przyspieszenie: 2,45x
Testowanie z 8 wątkami... Fine: 12090 ms, Coarse: 45743 ms, Przyspieszenie: 3,78x
Testowanie z 16 wątkami... Fine: 16506 ms, Coarse: 129162 ms, Przyspieszenie: 7,83x

----------------------------------------------------------------------
Koszt operacji: 10 ms
----------------------------------------------------------------------
Testowanie z 1 wątkami... Fine: 5822 ms, Coarse: 5801 ms, Przyspieszenie: 1,00x
Testowanie z 2 wątkami... Fine: 9334 ms, Coarse: 14361 ms, Przyspieszenie: 1,54x
Testowanie z 4 wątkami... Fine: 14419 ms, Coarse: 36415 ms, Przyspieszenie: 2,53x
Testowanie z 8 wątkami... Fine: 21828 ms, Coarse: 93054 ms, Przyspieszenie: 4,26x
Testowanie z 16 wątkami... Fine: 31294 ms, Coarse: 361270 ms, Przyspieszenie: 11,54x
```

5. Wnioski

Problem czytelników i pisarzy

- **Skalowanie nie jest liniowe.** Wykresy 3D pokazują, że wzrost liczby wątków nie przekłada się proporcjonalnie na wzrost czasu wykonania. Przy małej liczbie pisarzy i dużej liczbie czytelników, czas wykonania rośnie wolniej niż przy odwrotnej proporcji, co jest zgodne z założeniami problemu (czytelnicy mogą działać równocześnie). Zatem większy wpływ na wydajność ma liczba pisarzy, co jest logiczne.

Blokowanie drobnoziarniste

- **Koszt operacji jest kluczowy.** Przy zerowym lub minimalnym koszcie operacji (0-1ms) lista z jednym zamkiem jest szybsza ze względu na mniejszy overhead zarządzania wieloma zamkami. Przyspieszenie fine-grained przy koszcie 0ms wynosi zaledwie 0.26-0.40x.
- **Fine-grained wygrywa przy kosztownych operacjach.** Przy koszcie 10ms i 16 wątkach fine-grained osiąga **11.54x przyspieszenie** względem coarse-grained. Im droższe operacje, tym większa korzyść z równoległego dostępu do różnych fragmentów listy.

6. Bibliografia

https://pl.wikipedia.org/wiki/Problem_czytelnik%C3%B3w_i_pisarzy